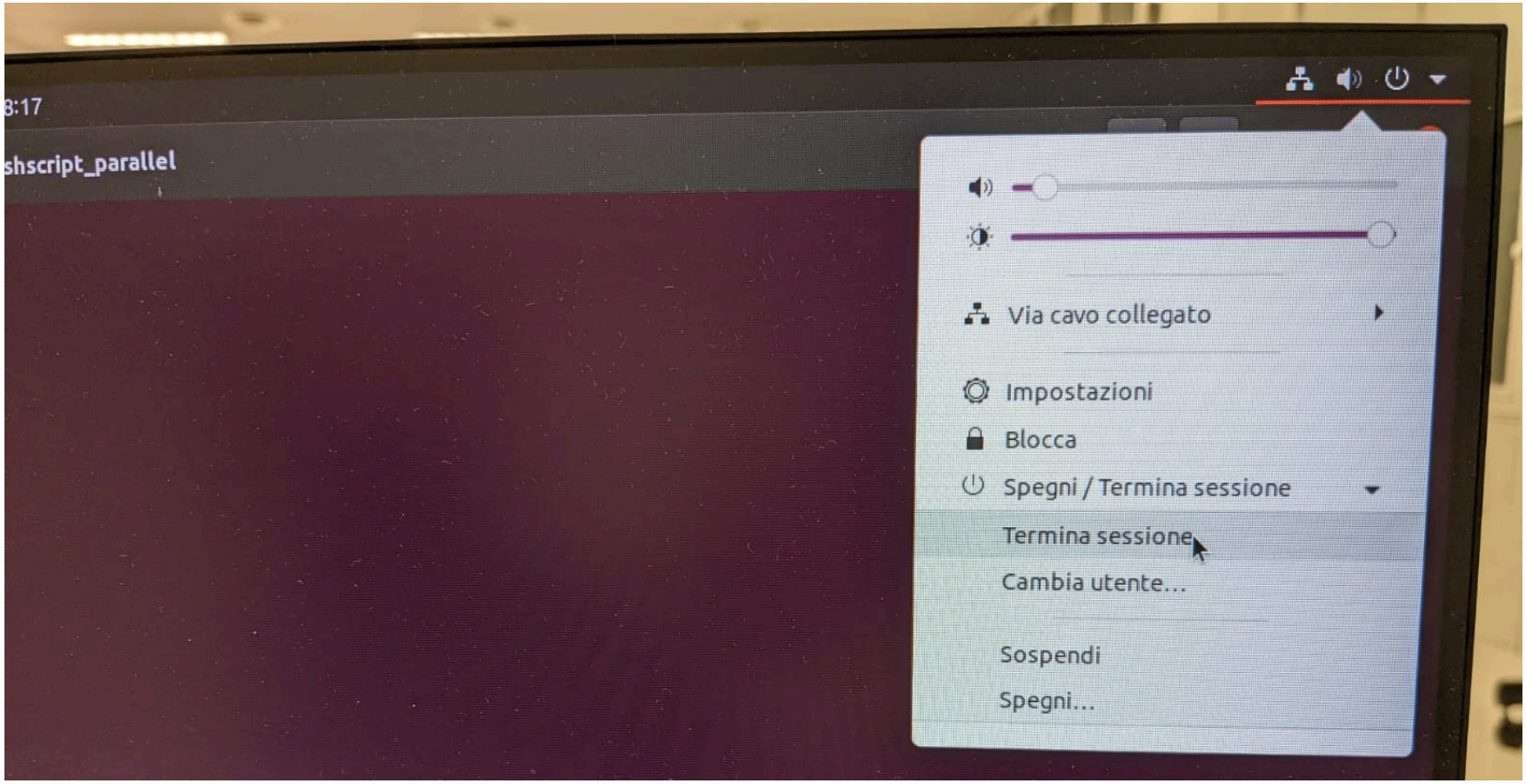
Corso di Sistemi Operativi

Corso estinto di Sistemi Operativi e Reti - Modulo Sistemi Operativi

Prova scritta - Luglio 2026

LEGGI ATTENTAMENTE LE ISTRUZIONI:

1. **Rinomina** subito la cartella chiamata "Cognome-Nome-Matricola" che hai trovato sul Desktop e in cui hai trovato questa traccia, sostituendo "Cognome" "Nome" e "Matricola" con i tuoi dati personali e **lasciando i trattini senza spazi**; se hai un doppio nome oppure un doppio cognome dovrai chiamare la cartella come in questo esempio:
    a. DeLuca-MarcoGiovanni-199999
2. **Carica** tutto il materiale didattico che vorrai usare sul Desktop; puoi farlo solo nei primi 5 minuti della prova;
3. **Svolgi** il compito; lascia tutto il sorgente che hai prodotto nella cartella di cui al punto 1;
4. Quando hai finito fai "**Termina Sessione/Logout**", ma lascia la postazione mantenendo il PC acceso.
5. **E' tua diretta responsabilità** garantire l'integrità del tuo elaborato, anche in caso di assenza di corrente. **Salva spesso il tuo lavoro**



e NON spegnere il PC.

SALVA SPESSO

CI SONO DEI PUNTI AMBIGUI NELLA TRACCIA? DECIDI TU COSA FARE

È parte integrante di questo esercizio completare le specifiche date nei punti non esplicitamente definiti, introducendo nuove strutture dati, o estendendo quelle preesistenti laddove si ritenga necessario, risolvendo eventuali ambiguità. Si può cambiare il codice dei metodi esistenti dove serve.

POSSO CAMBIARE IL PROTOTIPO DEI METODI RICHIESTI O DI QUELLI ESISTENTI? Si, ma solo se li lasci retrocompatibili

Il codice che scriverai è pensato per essere usato come un modulo di libreria. Un modulo di libreria potrebbe essere usato da altri programmatori, i quali si aspettano di trovare una specifica interfaccia.

Se quindi modifichi il prototipo dei metodi pubblici di una classe, devi assicurarti che possano essere comunque utilizzati per come erano stati pensati prima delle tue modifiche. Puoi aggiungere qualsivoglia campo e metodo privato, e qualsivoglia classe ausiliaria. Analogamente, i metodi esistenti possono essere modificati nel loro codice, ma non se ne deve cambiare il risultato finale o il significato a meno che non ti venga richiesto esplicitamente.

CHE LINGUAGGIO POSSO USARE? PYTHON 3.X

POSSO CONSENTIRE SITUAZIONI DI RACE CONDITION NEL MIO CODICE? NO
POSSO CONSENTIRE SITUAZIONI DI DEADLOCK NEL MIO CODICE? NO
POSSO CONSENTIRE ALTRE SITUAZIONI DI BLOCCO TOTALE NEL MIO CODICE, TIPO NESTED LOCKOUT, LIVELOCK O ALTRO? NO
POSSO CONSENTIRE SITUAZIONI DI STARVATION NEL MIO CODICE? SI, tranne quando ti viene chiesto esplicitamente di rimuoverle
MA IL MAIN() LO DEVO AGGIORNARE? E I THREAD DI PROVA? SI

E' obbligatorio implementare esplicitamente il codice di prova oppure modificare il codice di prova pre-esistente, e accertarsi che giri senza errori prima della consegna.

ESERCIZIO 1 - PROGRAMMAZIONE MULTITHREADED

(Punteggio minimo richiesto 18/30. Pesa per ⅔ del voto finale)

Svolgi tutti i punti della traccia in un unico file che chiamerai `SOLUZIONE.py`
`-> Qualsiasi altro nome di file verrà IGNORATO <-`

In questo appello dovrai progettare un simulatore di lavanderia a gettone sfruttando il codice del thread pool che ti abbiamo fornito. L'idea di base è quella di utilizzare dei `ThreadPool` per simulare le lavatrici e le asciugatrici. Ogni thread facente parte di un pool corrisponde a una lavatrice o ad una asciugatrice: così facendo potresti sfruttare l'idea di avere un Pool che esegua i job delle lavatrici e un pool di thread che esegua i job delle asciugatrici. I panni sono rappresentati da array di caratteri: `'*'` rappresenta un panno sporco; `'O'` rappresenta un panno pulito ma bagnato; `'-'` rappresenta un panno pulito e asciutto. Per lavare un panno sporco ci vogliono 100 millisecondi, mentre per asciugare un panno bagnato ci vogliono 50 millisecondi. Lavatrici e asciugatrici possono lavare/asciugare un intero carico per volta.

Punto 1
Progetta la classe `Lavanderia`. Una lavanderia dispone di `N` lavatrici e `M` asciugatrici. La `Lavanderia` deve disporre dei metodi `lava(self,carico)` e `asciuga(self,carico)`. Un `carico` è un array di caratteri definito come ti ho spiegato sopra. Sia `lava` che `asciuga` si comportano così: sottopone l'operazione di lavaggio/asciugatura rispettivamente al pool di lavatrici o al pool di asciugatrici; scrive a video quando l'operazione effettivamente inizia; scrive a video quando l'operazione effettivamente termina; infine esce dalla funzione.

Punto 2
Prevedi il metodo `lava_asciuga(self,carico)`, che combina le due operazioni e termina quando un carico di panni sporchi viene lavato e quindi asciugato.

Punto 3
Supponi che le lavatrici abbiano un carico massimo di 10kg ciascuna, e che ogni capo di abbigliamento pesi 1kg. Devi risolvere a tuo piacimento il problema del lavaggio di carichi superiori a 10kg.

Punto 4 [OPZIONALE]

Estendi la classe `ThreadPool` modificando il metodo `submit`, che diventa `submit(self,job1,job2 = None)`. Se `job2 = None`, `submit` si comporta come sempre; se `job2` è valorizzato, `job1` viene accodato per l'esecuzione ma **non può essere eseguito se** job2 non è ancora terminato.

SALVA SPESSO

ESERCIZIO 2 - LINGUAGGI DI SCRIPTING

(Punteggio minimo richiesto 18/30. Pesa per ⅓ del voto finale)

Trovi allegato a questa traccia uno script Perl che presenta errori di sintassi e/o logici. Sistemalo per ottenere il comportamento che ti descriviamo sotto. ***NON devi stravolgere il contenuto dello script o riscriverlo da capo: la sua struttura deve rimanere pressoché invariata. E' prevista una penalità se correggi parti di codice che invece non presenta bug.***

Lo script `diskUsage.pl` dovrebbe essere capace di filtrare determinate tipologie di file specificate dall'utente e di produrre in output un riassunto dettagliato dello spazio occupato su disco da quelle specifiche tipologie di file.

Lo script riceve in input i seguenti argomenti:

1. **(OBBLIGATORIO)** una lista di possibili formati di file (e.g., **pdf**,**txt**,**docx**, ...) e
2. **(OPZIONALE)** il path ad una cartella di sistema (e.g., **/home/esame/Scrivania/NomeCognomeMatricola**). Se l'utente non inserisce alcun path, di default, lo script effettuerà la ricerca all'interno della cartella corrente.

Lo script dovrà essere eseguito, quindi, con la seguente sintassi:

```
perl diskUsage.pl --formats={FORMATS} [path/to/directory]
```

Una volta eseguito, lo script dovrà controllare se, all'interno del path specificato (o nella cartella corrente), sono presenti file con le estensioni specificate tramite il parametro `--formats={FORMATS}`. Quindi, dovrà aggregare per estensione (ovviamente solo quelle specificate) i file presenti e sommare il valore complessivo di Kb occupati da ogni formato di file.

Prima di terminare, lo script stamperà su `STDOUT` l'elenco dei formati dei file filtrati, ciascuno dei quali associato ai rispettivi valori di uso dello spazio sul disco, e ordinati in ordine decrescente di valore. Infine, si stamperà su un `FILE` dal nome `du.out` la somma totale dello spazio su disco utilizzato dai file precedentemente filtrati.

ESEMPIO:

1. Contenuto della cartella SettembreTest :

```
SettembreTest/            6824
├── Sotto cartella        6784
│   ├── file1.pdf         5436
│   ├── file2.pdf         728
│   └── file3.pdf         620
├── diskUsage.pl          4
├── esempio.txt           8
├── main.pdf              0
├── output.a              0
├── test - Copia.docx     16
├── test.docx             12
└── txt.ciao              0
```

2. Esecuzione Script:

```
./diskUsage.pl --format=pdf,txt,docx ./LuglioTest
```

3. Azioni effettuate internamente dallo script

Lo script terrà in considerazione solo i file le cui estensioni sono **pdf , txt** oppure **docx.** Effettuerà per ognuna di queste estensioni la somma dei Kb occupati su disco e, infine, procederà con le stampe.

4.1. Output (STDOUT):

```
Estensione: pdf      6784Kb
Estensione: docx     28Kb
Estensione: txt      8Kb
```

4.2 Output (du.out):

```
Totale occupazione disco della cartella
./LuglioTest:  6820 Kb
```